ULB UNIVERSITÉ LIBRE DE BRUXELLES

EPFC

PRM2
Théorie : Leçon 13

Java :
Introduction aux méthodes

Méthodes d'instance

Vous avez déjà utilisé des méthodes d'instance :

`it.next()` appelle la méthode `next()` sur le `SeqIntIterator` référencé par `it`.

`tab.set(idx, val)` appelle la méthode `set(idx, val)` sur le `Array` référencé par `tab`.

Une **méthode d'instance** est donc une méthode qui s'appelle à l'aide d'un **point** sur un **référent** et **manipule l'objet référencé**.

Ces méthodes ne portent pas le modificateur `static` et sont définies dans la classe en question.

Elles définissent le comportement des instances de cette classe.

Définition et appel de méthodes

Une méthode d'instance est définie dans la classe dont elle manipule les objets. Elle ne porte pas le modificateur `static` et s'appelle à l'aide d'un point sur un référent.

```
public class Programme {
    public static void main(String[] args) {
        Personne p = new Personne();
        p.saisie();
        p.affiche();
    }
}
```

```
public class Personne {
    public String nom, prenom;

    public void affiche() {
        System.out.println(this.nom + " " + this.prenom);
    }

    public void saisie() {
        Scanner in = new Scanner(System.in);
        this.nom = in.next();
        this.prenom = in.next();
    }
}
```

Dans le corps d'une méthode d'instance, le mot clé `this` représente l'objet courant. C'est à dire l'objet sur lequel a été appelé la méthode.

Fonctions statiques et méthodes d'instance

Pour faire simple, toute fonction statique qui reçoit et manipule un **objet particulier** d'une classe devrait être une méthode d'**instance** (non statique) de cette classe.

Comparez les deux :

```java
public class Personne {
    public String nom, prenom;

    public void affiche() {
        System.out.println(this.nom + " "
                           + this.prenom);
    }
}

public class Programme {
    public static void main(String[] args) {
        Personne p = new Personne();
        //...
        p.affiche();
    }
}
```

```java
public class Personne {
    public String nom, prenom;
}


public class Programme {

    public static void affiche(Personne p) {
        System.out.println(p.nom + " "
                           + p.prenom);
    }

    public static void main(String[] args) {
        Personne p = new Personne();
        //...
        affiche(p);
    }
}
```

La méthode d'instance `String toString()`

Vous l'avez remarqué : lorsque l'on passe un objet à `println()`, celui-ci affiche son type et son adresse :

`System.out.println(p)` affiche par exemple `Personne@7852e922`

Si vous définissez la méthode d'instance `public String toString()` dans une classe, `println()` l'utilise pour produire une représentation textuelle de l'objet :

```java
public class Personne {
    public String nom, prenom;

    //...

    public String toString(){
        return this.nom + " " + this.prenom;
    }
}
```

```java
public class Programme {
    public static void main(String[] args) {
        Personne p = new Personne();
        p.nom = "Dupont";
        p.prenom = "Wilbur";
        System.out.println(p);
    }
}
```

Le programme affiche
`Dupont Wilbur`

Constructeurs

Plutôt que d'écrire

```
Personne p = new Personne();
p.nom = "Dupont";
p.prenom = "Wilbur";
```

et, comme chaque personne possède un nom et un prénom, vous voudrions écrire

```
Personne p = new Personne("Dupont", "Wilbur");
```

C'est le but d'un constructeur.

Un constructeur est une méthode spéciale servant à donner une première valeur aux attributs d'un objet. Il est appelé lors de la création d'un objet avec new.

Définition de constructeurs

Un constructeur est une méthode spéciale permettant, notamment, de donner une première valeur aux attributs d'un objet lors de sa création avec new. Il porte le même nom que la classe, n'a pas de type de retour et ne renvoie rien.

```
public class Personne {
    public String nom, prenom;

    public Personne(String nom, String prenom) {
        this.nom = nom;
        this.prenom = prenom;
    }
}
```

A partir du moment où un constructeur est défini dans une classe, il est obligatoire de l'utiliser. Avec le code ci-dessus, il n'est donc plus possible de construire une personne à l'aide de `new Personne()` (sans argument).

Si, en plus, un constructeur sans argument est nécessaire, il faut le définir :

```
public Personne() { }
```

Remarquez qu'ainsi, une classe peut posséder plusieurs constructeurs.